

WorkFlow Pro – Multi-Platform Test Automation Assessment

Name: parth chilwerwar

Email: job.parthchilwerwar@gmail.com

Date: Jul 20, 2026

Assumptions

Because some requirements are intentionally incomplete, I made the following assumptions:

- A dedicated test environment and test accounts are available.
- Test users do not use real customer data.
- The Admin test account either has 2FA disabled or supports a test-only OTP mechanism.
- Every tenant has a unique tenant ID and domain.
- The API supports deleting test projects after execution.
- The web application provides stable test selectors such as data-testid.
- The mobile requirement refers to the responsive web application running on mobile browsers. If WorkFlow Pro also has a native mobile app, I would use Appium or BrowserStack App Automate instead of Playwright.
- BrowserStack credentials are stored as CI secrets and are never committed to the repository.

Part 1: Debugging the Flaky Login Tests

1. Problems in the Existing Tests

The main problem is that the tests assume the application responds immediately. That may happen locally, but it is unreliable in CI.

Immediate URL assertion

```
page.click("#login-btn")
```

```
assert page.url == "https://app.workflowpro.com/dashboard"
```

The click starts the login request, but the test checks the URL immediately afterward. The authentication request, redirect and dashboard loading may still be in progress. This creates a race condition.

Exact URL comparison

The test assumes that the final URL will always be exactly: `https://app.workflowpro.com/dashboard`. It may instead contain a trailing slash, query parameters, a tenant-specific domain, a redirect parameter, a first-login page, or a 2FA verification page. The login may therefore succeed while the assertion still fails.

Non-retrying visibility check

```
assert page.locator(".welcome-message").is_visible()
```

`is_visible()` checks the current state. It does not keep retrying until the dashboard finishes rendering. A retrying Playwright assertion is more reliable:

```
expect(page.locator(".welcome-message")).to_be_visible()
```

Dynamic dashboard loading is ignored

The dashboard elements are loaded dynamically. Waiting only for navigation does not guarantee that project data or the welcome message has finished loading. The test should wait for a meaningful application state.

No handling for 2FA

Some users may be redirected to a verification page. The current test assumes every account reaches the dashboard directly.

Weak selectors

Selectors such as `#email`, `#password`, `#login-btn`, `.welcome-message` are tightly connected to implementation details. I would prefer accessible or test-specific selectors like `page.get_by_test_id("welcome-message")`.

Using `.all()` too early

```
projects = page.locator(".project-card").all()
```

This collects the elements that exist at that exact point. If the projects are still loading, the returned list may be empty.

Tenant validation is based only on visible text

Tenant isolation should instead be verified using known test project IDs or unique project names. It should also be validated through the API, not only through visible text.

Hard-coded credentials

Credentials should come from environment variables, CI secret storage, or a secure test configuration service.

2. Why It Passes Locally but Fails in CI

Local environments are usually faster and more stable. CI environments are less predictable due to parallel execution, limited resources, cold cache, network variance, and different browser behaviors. The root problem is that the test depends on timing instead of waiting for observable application states.

3. Corrected Test Implementation

Python

```
# conftest.py
import os
import re
import pytest
from playwright.sync_api import Browser, BrowserContext, Page, expect

BASE_URL = os.getenv("WORKFLOWPRO_BASE_URL",
"https://app.workflowpro.com")

@pytest.fixture(scope="session")
def base_url() -> str: return BASE_URL

@pytest.fixture
def context(browser: Browser) -> BrowserContext:
    context = browser.new_context(viewport={"width": 1440,
"height": 900})
    context.set_default_timeout(15_000)
    context.set_default_navigation_timeout(30_000)
    yield context
    context.close()

@pytest.fixture
def page(context: BrowserContext) -> Page:
    page = context.new_page()
    yield page
```

Python

```
# Reliable login helper
import os
import re
from playwright.sync_api import Page, expect

def login(page: Page, base_url: str, email: str, password: str)
-> None:
    page.goto(f"{base_url}/login", wait_until="domcontentloaded")
    page.get_by_label("Email").fill(email)
    page.get_by_label("Password").fill(password)
    page.get_by_role("button", name=re.compile("sign in|login",
re.I)).click()

    # Handle optional 2FA
    two_factor_input = page.get_by_label(re.compile("verification
code|otp", re.I))
    if two_factor_input.is_visible(timeout=3_000):
        two_factor_input.fill(os.getenv("WORKFLOWPRO_TEST_OTP"))
        page.get_by_role("button",
name=re.compile("verify|continue", re.I)).click()

    page.wait_for_url(re.compile(r".*/dashboard(?:/|\?\.*)?$"),
timeout=30_000)

expect(page.get_by_test_id("dashboard")).to_be_visible(timeout=20
_000)
```

Part 2: Test Framework Design

1. Proposed Framework Structure

None

workflowpro-tests/

```
|— config/
|— tests/
| |— web/
| |— api/
|— pages/
|— components/
|— api/
|— fixtures/
|— utils/
```

2. Design Approach

The framework is divided into four main layers:

- **Test layer:** Describes business behaviour.
- **Page-object and component layer:** Encapsulates UI behaviour.
- **API client layer:** Centralizes backend communication and validation.
- **Fixture layer:** Manages setup and teardown of environments, sessions, and data.

3. Role-Based Testing

Permissions should be verified by running parameterized tests that check both UI visibility and backend API authorization (i.e., making sure you get a 403 Forbidden error).

Part 3: API + UI Integration Test

1. Testing Strategy

Setup the data via API—it's faster and more reliable—and then use the UI to validate everything.

2. Full Integration Test (Snippet)

Python

```
@pytest.mark.integration
def test_project_creation_flow(playwright: Playwright, company1,
company2):
    # Step 1: Create the project through the Company1 API.
    project = company1_api.create_project(project_data)
    project_id = project["id"]

    # Step 2: Verify the project in a desktop browser.
    login(company1_page, company1.web_base_url, company1.email,
company1.password)
    projects_page.expect_project_visible(project_id,
project_data.name)

    # Step 3: Validate Company2 cannot see the project.
    company2_projects.expect_project_not_visible(project_id,
project_data.name)

    # Step 4: Cleanup
    company1_api.delete_project(project["id"])
```

3. Final Testing Approach

- Wait for the business state to be ready, don't just add wait times.
- Use APIs for fast setup and cleanup.
- Keep tenant/user data isolated.
- Validate authorization at both the UI and API levels.
- Capture enough evidence (screenshots, logs) so you can investigate failures without having to rerun the tests manually.